



BugMirror

照妖镜

编译 / 测试缺陷智能知识库

洞察每一笔缺陷背后的真相

语义搜索

知识沉淀

AI 根因分析

RAG 问答

为什么需要 照妖镜 ？

在大规模编译与测试场景下，缺陷排查面临三大核心痛点



知识碎片化

缺陷经验散落在日志、工单、IM 对话中，无法沉淀和复用。新人遇到相同问题只能重新排查，老工程师的经验无法传承。

⚡ 平均每个缺陷重复排查 2~3 次



搜索效率低

传统关键词搜索无法理解问题语义，稍有措辞差异就搜不到。同一个 root cause 在不同环境下错误信息完全不同。

🔍 关键词搜索召回率不足 30%



排查周期长

面对陌生缺陷，工程师缺乏系统化的分析工具，只能凭经验逐步试错。根因定位过程漫长而痛苦。

🕒 平均修复时长超过 12 小时

BugMirror 的目标：让缺陷排查从 “重复踩坑” 变成 “知识驱动”

产品定位与核心价值

面向大规模编译、测试与运行时缺陷的智能知识库系统。
通过语义检索与大模型增强生成（RAG），让团队把踩坑变成知识驱动。

 **语义级缺陷检索**

基于向量相似度理解问题本质，
不依赖关键词

 **AI 辅助根因分析**

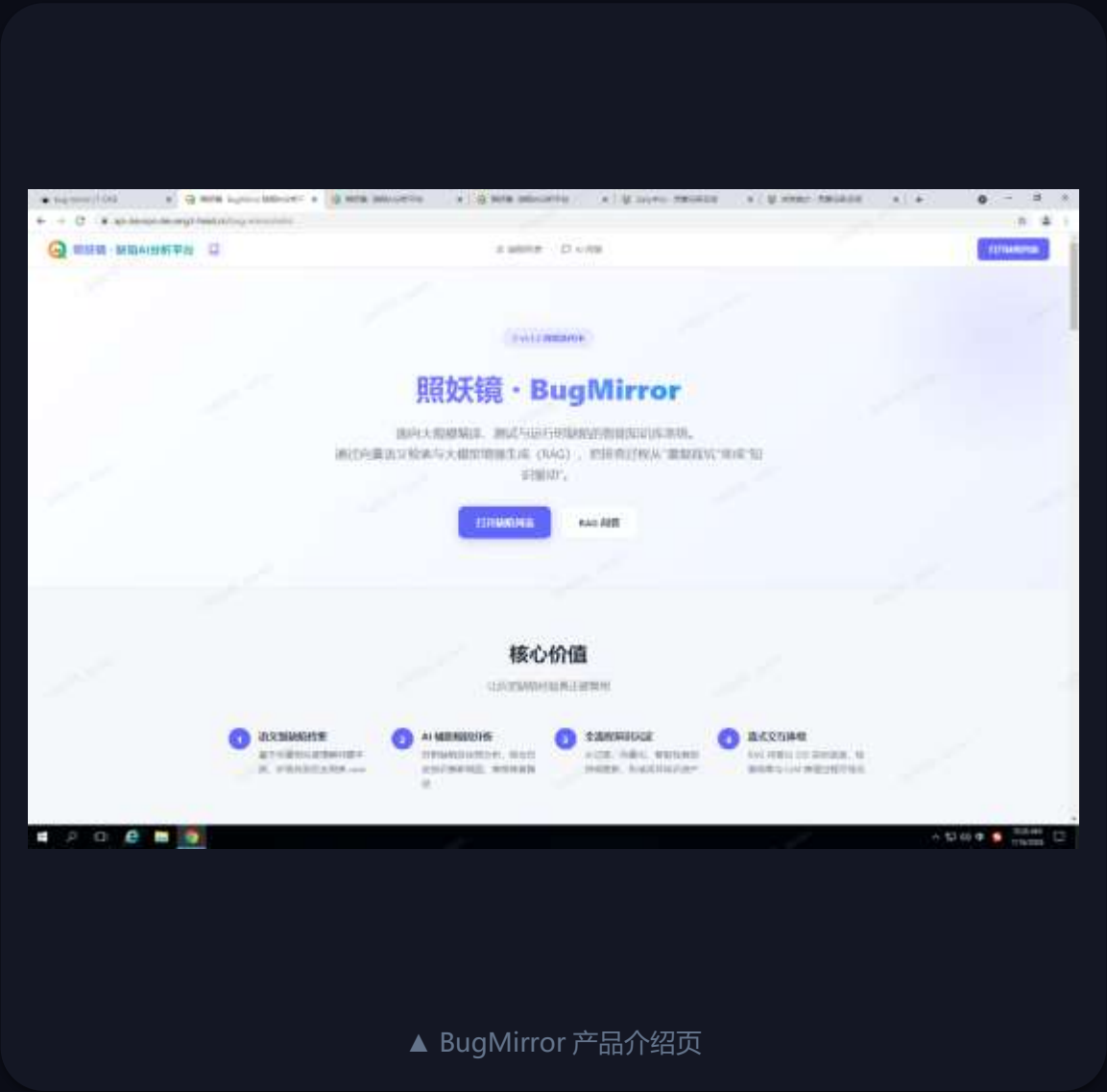
自动预分析，推断根因、推荐修复路径

 **全流程知识沉淀**

从记录到检索到更新，形成闭环知识资产

 **流式交互体验**

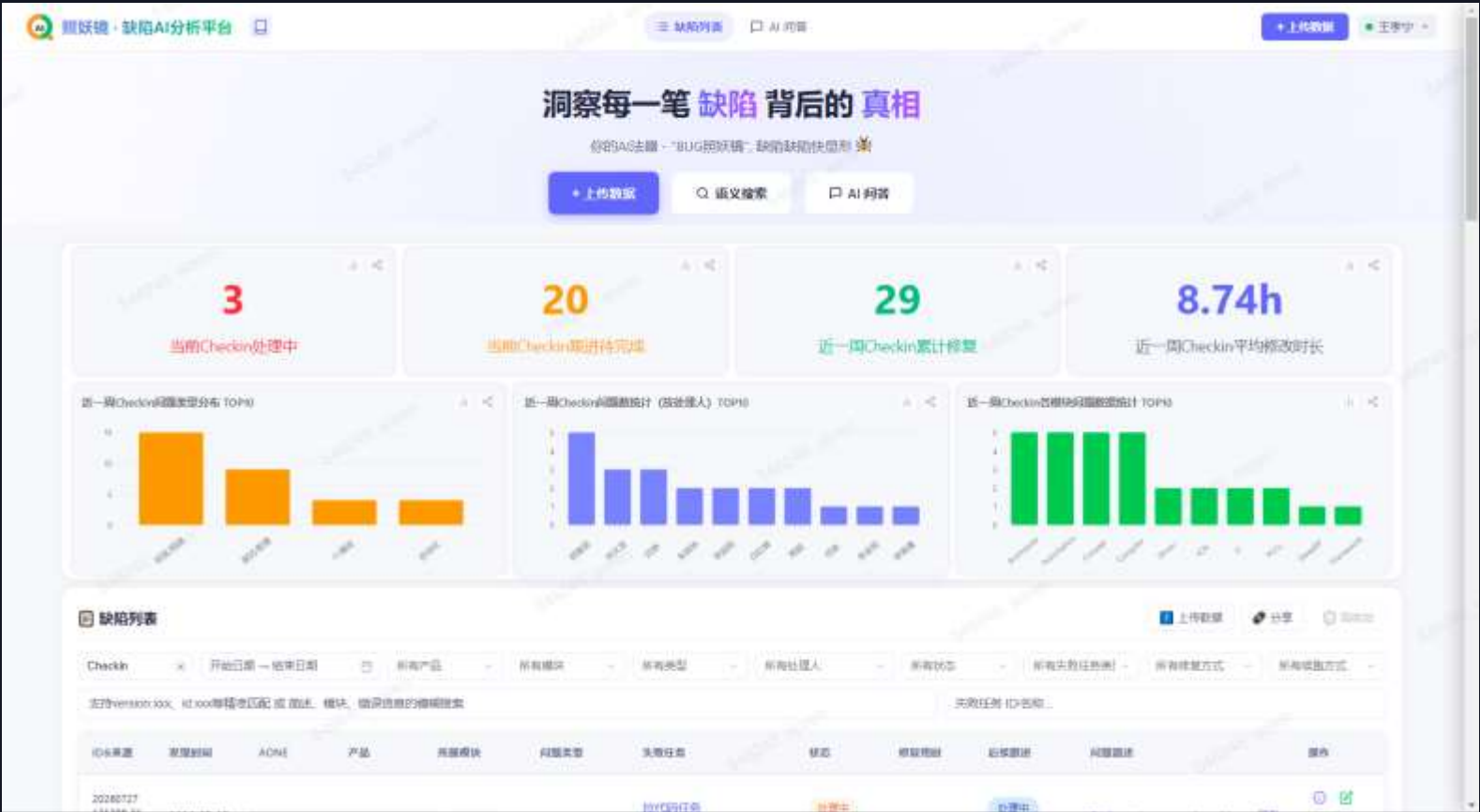
RAG 问答 SSE 实时返回，过程可视可追溯



▲ BugMirror 产品介绍页

主页总览 — 数据一目了然

实时统计卡片 + 多维 TOP10 图表 + 缺陷列表，一页掌握全局态势



实时统计

处理中 / 待完成 / 累计修复 / 平均时长

TOP10 图表

问题类型分布 · 处理人排行

缺陷列表

可筛选 · 可搜索 · 可排序 · 可编辑

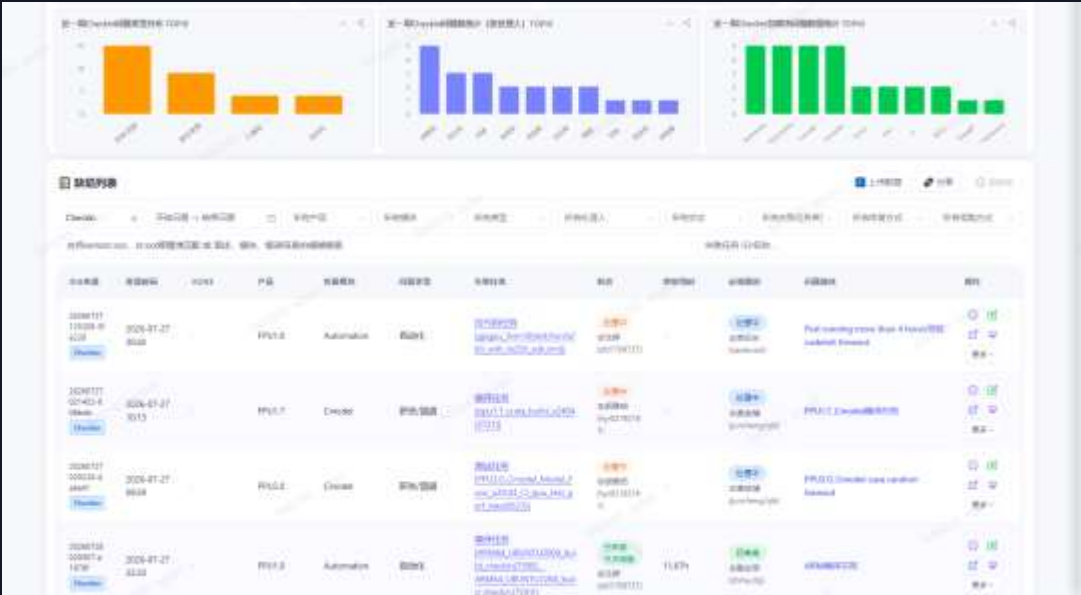
一键操作

AI 分析 · 编辑 · 语义搜索 · 问答

缺陷管理 — 全链路追踪

从列表到详情，支持多维筛选、状态追踪和进度可视化

- 缺陷列表 — 状态进度条 + 完整字段表格



- Checkin 处理中详情



- 跟进待完成详情

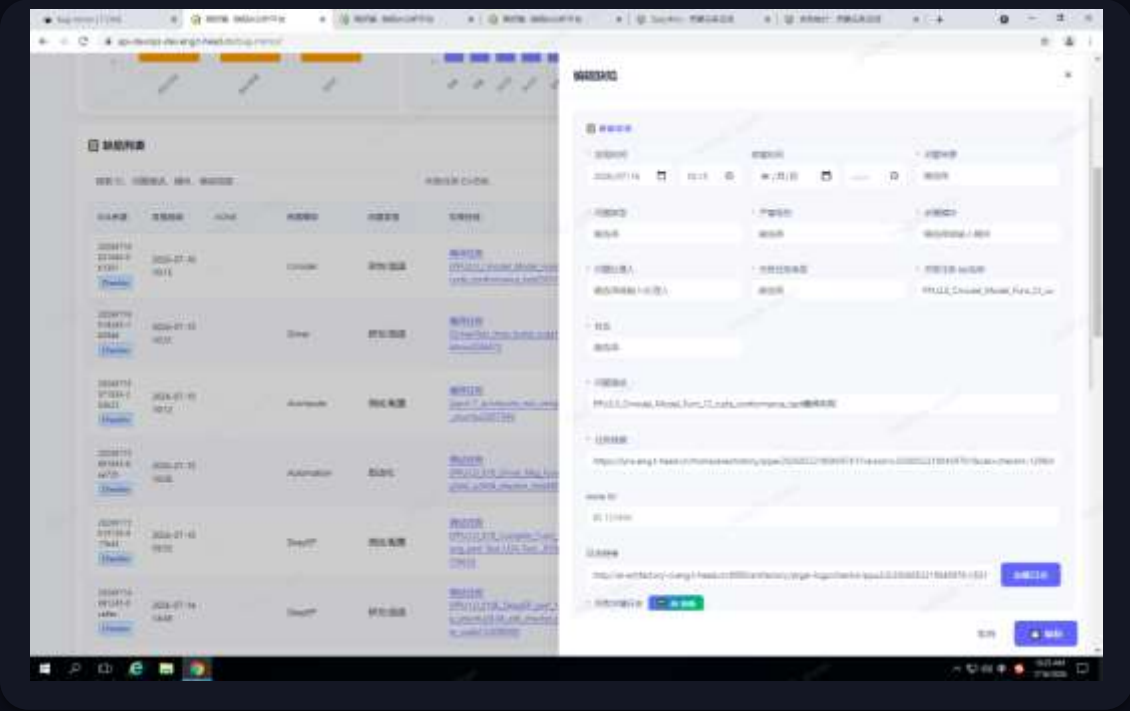


亮点：顶部彩色状态进度条直观展示各状态占比；每行支持一键 AI 分析、编辑操作；支持按 ID / 描述 / 模块 / 错误信息等多维度组合筛选

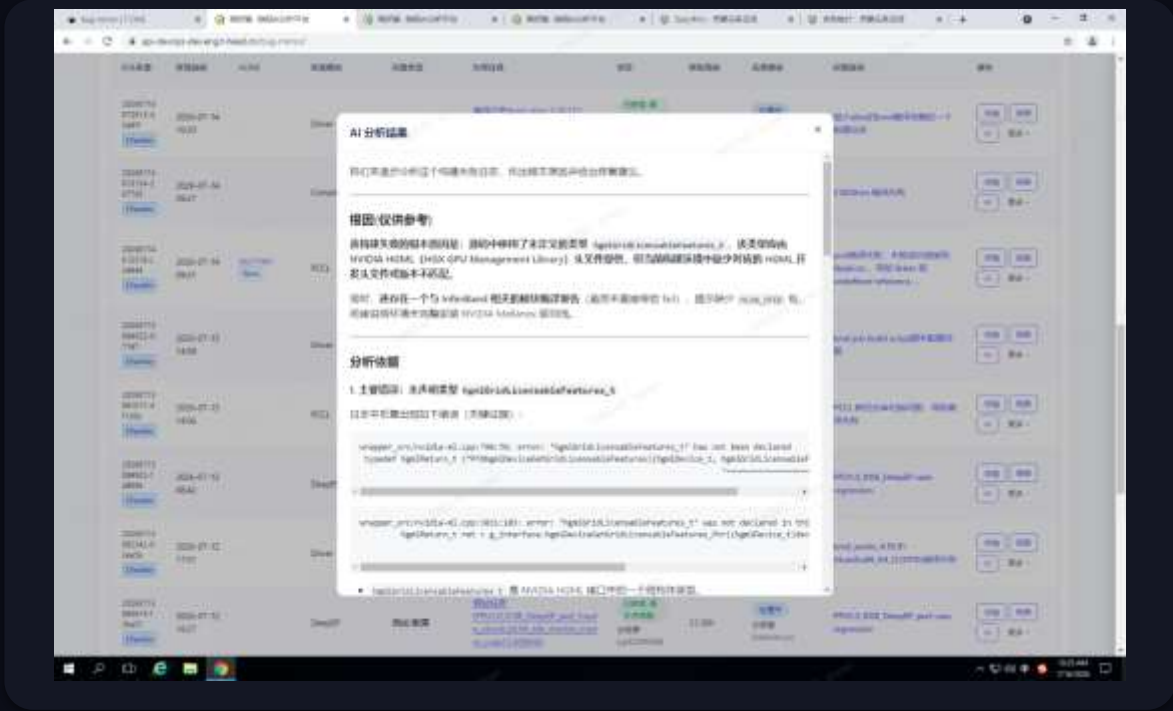
缺陷录入与 AI 预分析

结构化录入 + 一键 AI 分析，让每一条缺陷记录都自带根因推理

● 右侧抽屉表单 — 结构化录入



● AI 分析结果 — 自动根因推理



必填校验

发现时间 / 问题描述 / 来源 / 类型

分类维度

所属模块 / 严重级别 / 失败任务

关联信息

CI 链接 / AONE ID / 日志链接

错误信息

支持粘贴完整错误日志文本

AI 分析能力：

- ① 自动提取错误日志中的关键信息
- ② 给出根因推断（仅供参考）
- ③ 提供详细的分析依据，含代码级错误定位
- ④ 结合历史知识库中的相似案例辅助判断

语义搜索 — 理解问题本质

不再依赖关键词精确匹配。输入自然语言描述，基于向量相似度秒级返回历史中最相似的缺陷。

▶ 自然语言查询

直接输入问题描述，如「泛型类型推断错误」

▶ 多维度过滤

可选择关注类型（全文 / 仅标题 / 仅错误信息）

▶ 按 ID 找相似

对已有缺陷一键查找历史同类 case

▶ 相似度排序

结果按 cosine 相似度降序，最相关排最前

✗ 传统关键词搜索

undefined reference ≠ 未定义引用
措辞不同 → 搜不到

✔ BugMirror 语义搜索

理解语义本质，“未定义引用”
也能匹配 undefined reference

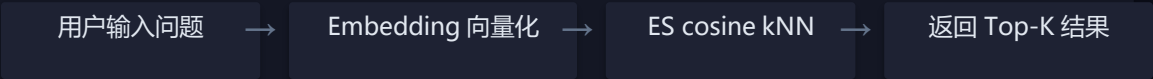
💡 搜索示例

自然语言查询
编译报错 **undefined reference to template**
按 ID 找相似
--similar compile-001



▲ 语义搜索弹窗

搜索流程



AI 问答 — 知识增强生成

基于 RAG (检索增强生成) 架构，结合历史缺陷知识库，提供精准问题诊断与修复建议



▲ AI 问答交互页面

1

用户提问

输入编译错误、测试失败或运行时问题描述

2

语义检索

向量化问题 → ES kNN 检索 → 返回 Top-K 相似缺陷

3

LLM 增强生成

检索结果作为上下文 → LLM 推理 → SSE 流式输出

4

结果展示

检索来源可追溯 + 降级策略 (LLM 不可用时回退纯检索)

SSE 流式

来源可追溯

降级兜底

REST API

MCP 生态集成 — 百灵 AI + DevOps CLI

通过 MCP (Model Context Protocol) 协议，将 BugMirror 知识库能力无缝接入 AI 助手和命令行工具链

DevOps CLI 集成

在终端中直接调用 BugMirror MCP 搜索缺陷知识库，返回结构化结果并自动执行修复命令

```
(base) allan@ml001:~/ml10$ ls -la
(base) allan@ml001:~/ml10$ cd /home/allan/bugmirror
(base) allan@ml001:~/bugmirror$ python3 search.py --collect 21 --error 1 --exit status 2026-07-28 01:45:02.834256 jppu_classification error: http test linker command failed with exit code 1 (use -v to see invocation) 2026-07-28 01:45:02.834333 error: command 'mcc' failed with exit code 2' --pretty
{"results": [
  {
    "id": "20260728034121-ef0fed",
    "description": "PCU2.0_818E_DeepEP case random fail",
    "severity": "medium",
    "status": "fixed",
    "assignee": "程豪 (p02295488)",
    "found_datetime": "2026-07-28T11:31:00",
    "fix_datetime": "2026-07-28T11:46:40",
    "acorn": 0.0720,
    "problem_type": "测试/配置",
    "module": "DeepEP",
    "problem_classification": "用例充分验证",
    "issue_source": "Checkin",
    "task_link": "https://jira.eng.t-head.cn/home/eeddata/piper/20260728/4024474/cb-checkin-b07ef3e",
    "ownerid": "",
    "ownerstatus": null,
    "tag_link": "https://ai-tester.result.eng.t-head.cn/api/v1/core/oss/download/osskey=theis_test/2546041-4673511-c998b4e8b0411fa37fca2eadf734d3-ans0/2546041/4673511/19147387/17793120/1/run_log",
    "fail_test_type": "测试任务",
    "fail_test_id_name": "PCU2.0_818E_DeepEP_perf_inade_ubuntu24_04_sdk_checkin_master_sdk13_0188541",
    "retest_method": "重启设备",
    "duration_hours": 0.15,
    "root_cause": "测试case已暂时下线，问题待查中",
    "follow_up_assignee": "杨勇 (f1ee1ee.yv)",
    "follow_up_status": "处理中",
    "follow_up_measure": "",
    "creator": "沈野",
    "error_message_highlight": "run command: bash /pexu/share/pcc/perf/run_qpt_DeepEP_anales.sh DEEPER_USE_JIT=MMU_SWMMU_ENV python test_low_latency.py --daily --num-experts 128 --hidden-size 4096 --num-per-node MMU_SWMMU_ENV --num-tokens 0 --run-logk 0 --ll-vadn-zen MMU_SWMMU_ENV --without-barrier, current path: /home/allan/bugmirror/testcase"
  }
]
```

语义搜索

自然语言或日志片段查询，返回匹配缺陷及相似度分数

结构化输出

返回 JSON 格式结果：ID、描述、严重度、根因、修复方案

一键修复

直接在终端执行推荐的修复命令，如重新编译、安装依赖等

百灵 AI 对话集成

在百灵 AI 对话界面中直接查询 BugMirror 知识库，获取智能诊断和修复建议



知识库问答

粘贴报错信息，AI 自动检索 BugMirror 找到匹配的历史缺陷记录

智能诊断

结合匹配结果给出问题描述、根因分析和修复方案的完整报告

对话式交互

支持多轮追问，逐步深入排查；来源可追溯，标注知识库记录 ID

技术架构 — 全栈概览

轻量、高效、可扩展的技术选型

🔧 系统工程 workflow



15+ REST API 端点

3 前端核心页面

SSE 流式问答协议

技术栈

后端框架	FastAPI + Uvicorn (异步高性能)
向量数据库	Elasticsearch dense_vector + cosine kNN
嵌入服务	OpenAI 兼容接口 外部 Embedding 服务
LLM 服务	OpenAI 兼容接口 可插拔 LLM Provider
前端	原生 HTML/CSS/JS (ES Module) 零框架、轻量极速
数据验证	Pydantic v2
日志 & 容器化	Loguru + Dockerfile

混合搜索算法 — kNN + BM25 + RRF + Token 校准

查询分类 → 日志归一化 → 双向量 kNN + 停用词 BM25 双路召回 → RRF 融合 → Token 重合率校准

① 混合搜索管线 (search)

A 查询分类 classify_query

多特征评分：多行(+2) / 文件路径(+2) / 构建工具(+2) / 错误关键词(+1) / 代码特征(+1) → 总分 >= 2 判定为日志

B 日志归一化 normalize_log

去时间戳 → 绝对路径保留末2级 → 十六进制地址 → <ADDR> → PID/TID移除 → 文件: 行号 → 仅文件名 → 压缩空行

C 双向量 kNN 搜索

日志查询：搜 vector_error(错误签名向量) + vector(全文向量) 取 max score；自然语言：仅搜 vector。HNSW 索引, num_candidates=top*10

D BM25 全文检索 + 停用词过滤

~300 停用词表(编译/C++/Java/日志级别等)先过滤再搜。日志：error_message^3/.code^5 + searchable_text^2；自然语言：均衡权重

E RRF 融合 + Token 重合率校准

RRF: score = $\sum 1/(60 + \text{rank}_i)$ 仅排序, 展示分 = kNN cosine 原值。日志查询额外校准: score = raw * (0.15 + 0.85 * overlap), overlap = query 与 error_message 的 token 重合率

min_score = 0.50 | 仅BM25命中展示分 = kNN最低分*0.8 | BM25高亮error_message

② BM25 停用词 & Token 重合率校准

两大抗噪机制：BM25 查询前过滤无区分度 token + RRF 后按关键词重合率校准排序

停用词表 (~300 词)

英语虚词 + 日志级别词 + 编译过程词 + C/C++/Java/Python关键字 + 系统路径 + 通用编程术语

→ Token 重合率校准 (日志查询专用)

overlap = $|Q \cap E| / |Q|$ (>=3 字符 token, 去停用词), score = raw * (0.15 + 0.85 * overlap), 惩罚伪相关结果

→ BM25 高亮 & 展示分策略

BM25 返回 error_message 高亮片段；仅BM25命中文档展示分 = kNN最低分*0.8, 由Token校准进一步调整

③ 日志预处理 — 错误签名提取

从完整错误日志中提取最有区分度的关键行，使相同类错误可靠匹配

堆栈模式

检测 Java/Python/C++ 堆栈特征
保留前5行（入口上下文）
保留后8行（根因在底部）
无条件保留 Caused by / Error 行

非堆栈模式（编译/链接日志）

优先保留含 error/fatal/failed 的行
其次保留含代码特征(;; ->) 的行
不够时补充上下文行
最终截断到 30 行 / 1500 字符

vector: 全字段拼接向量 | vector_error: 错误签名专用向量 → HNSW cosine 索引

RAG 问答算法 — 匹配判定 + Prompt 工程 + 流式生成

多层阈值匹配判定 → 检索结果组装为 RAG 上下文 → LLM 流式生成 → 降级兜底策略

④ RAG 问答服务管线 (QAService)

1

Embedding + 检索

问题向量化 → 召回 top_k 条候选 (当前未启用精排, 直接返回 kNN+BM25+RRF+Token校准 结果)

2

双重匹配判定 `_is_match`

策略1: top-1 分数 ≥ 0.45 (绝对阈值) → 命中
策略2: top-1 ≥ 0.25 且与 top-2 分差 ≥ 0.08 (区分度启发式) → 命中

3

RAG 上下文组装

命中时: 将候选缺陷的 `description/severity/error_message/root_cause` 编号格式化注入上下文
未命中时: `context = None`, LLM 基于通用知识回答

4

Prompt 策略 (双模板)

有匹配: 引导引用知识库记录 (#1, #2...), 严禁编造不存在的信息
无匹配: 先告知未命中, 再给出通用技术建议, 标注来源

5

SSE 流式输出 + 降级

事件流: `retrieval` → `token`(多次) → `done` | 异常降级: Embedding 不可用 → `error` 事件; LLM 超时 → `fallback` 到纯检索

`match` ≥ 0.45 | `related` ≥ 0.35 | `suggestion` ≥ 0.25 | `gap` ≥ 0.08

⑤ 返回结构与事件流

`found = True`

`primary`: 最佳匹配 (完整字段)
`related`: 关联缺陷列表
 `score > 0.35` 的记录
 最多返回 5 条

`found = False`

`suggestions`: 推荐候选列表
 `score > 0.25` 的记录
 最多返回 3 条
 仅含 `id/description/score`

⑥ ES 索引架构 & 分词策略

`dense_vector`

HNSW cosine 索引, 双向量: `vector`(全文) + `vector_error`(错误签名)

`text` 字段

三子字段结构: CJK bigram 分词 + `.code(word_delimiter)` + `.keyword`(精确)

自定义分析器

`bugmirror_code`: `standard` + `word_delimiter_graph`, 处理代码 `token`

WHERE 解析器

递归下降解析 SQL-like 过滤串 → 编译为 ES bool DSL (`=`, `>=`, `<=`, `LIKE`, `IS NULL`)

总结与未来展望

✦ 已实现能力

- ✓ **智能缺陷知识库**
结构化录入 + 向量化存储 + 全流程管理
- ✓ **语义级缺陷检索**
基于 Embedding + cosine kNN，告别关键词局限
- ✓ **AI 辅助分析 + RAG 问答**
一键根因分析 + 流式 AI 对话，检索来源可追溯
- ✓ **数据看板与行为分析**
实时统计 + 多维 TOP10 + 用户行为埋点
- ✓ **开放 API 与脚本化**
15+ REST 端点，可接入 CI/CD、告警平台

✦ 未来方向

- ◇ **CI/CD 深度集成**
编译失败自动触发录入 + 匹配历史 case + 推送修复建议
- ◇ **多模态错误分析**
支持截图 / 日志文件直接上传，OCR + LLM 联合理解
- ◇ **自动化闭环**
从发现到修复到验证，打通自动化 workflow
- ◇ **知识图谱构建**
缺陷之间的因果链、模块依赖关系可视化
- ◇ **团队协作增强**
评论 / @提及 / 缺陷订阅通知 / 周报自动生成

让缺陷排查从“重复踩坑”变成“知识驱动”

BugMirror 照妖镜 — 你的 AI 法器，把问题沉淀为经验